

Verdict: not ready to submit — but the code is not the problem.

The build is technically sound and passes the checks that most often cause rejection: no remote code, no bundled keys, correct Manifest V3 structure, complete icons. Publication is blocked by two policy issues — a missing privacy policy and a permission/scope combination that is likely to be refused — plus store listing assets that do not yet meet the required dimensions.

Findings at a glance

AREA	STATUS	DETAIL
Extension code	PASS	MV3, no remote code, no <code>eval</code> , 576 KB package
Icons	PASS	16, 32, 48, 128 px all present and declared
Secrets	PASS	No API key bundled; users supply their own
Automated tests	PASS	73 unit + 14 browser tests passing
Privacy policy	BLOCK	Does not exist; required because user data is handled
<code>debugger</code> permission	BLOCK	High rejection risk; not needed by the core feature
Single purpose policy	BLOCK	Three distinct products in one extension
Screenshots	FIX	None match the required 1280×800 or 640×400
Listing copy	FIX	Description, category and permission justifications needed
Licence file	FIX	No <code>LICENSE</code> in the repository

Blockers

1. No privacy policy BLOCKS SUBMISSION

The Web Store requires a published privacy policy whenever an extension handles user data. Formly does, in two ways: it stores an API key in local extension storage, and it transmits form field metadata (labels, names, placeholders, constraints) to Google or Groq.

You must publish the policy at a public URL and complete the data-use disclosures in the developer dashboard. The disclosures must match actual behaviour — state plainly that field labels leave the browser when AI is enabled, and that entered values never do.

2. The `debugger` permission **HIGH REJECTION RISK**

Reviewed far more strictly than any other permission in the manifest. It also shows a persistent browser banner — “*Formly started debugging this browser*” — while recording, which users report as alarming.

It exists solely to capture response bodies for the optional Requests inspector. That is an auxiliary feature, not the stated purpose of the extension, which is the case reviewers most often refuse. Removing it for the first submission removes the risk entirely.

3. Single purpose policy **HIGH REJECTION RISK**

Store policy requires “a single purpose that is narrow and easy to understand”. The current build ships three separable products:

- **A form filler** — the stated purpose.
- **A network request recorder** — captures a tab's submissions.
- **A request editor that replays authenticated requests** using the site's cookies and captured authorisation headers.

The third is effectively an HTTP client with credential reuse. Combined with `debugger`, a reviewer may classify the bundle as a security-testing tool rather than a form filler. Appeals are slow, so it is cheaper to avoid the classification than to argue it.

4. Broad optional host permissions **NEEDS JUSTIFICATION**

`http://*/*` and `https://*/*` are correctly declared as *optional* and requested at runtime, which is the right design. They still require a written justification in the listing, and they are only needed when “Prepare ahead of the click” is enabled. Say exactly that.

Store listing assets

Screenshot dimensions are enforced. Every current image needs replacing or padding:

FILE	CURRENT	REQUIRED	ACTION
docs/images/welcome.png	1280×794	1280×800	Pad 6 px
docs/images/generator.png	1280×960	1280×800	Re-shoot or crop
docs/images/sidepanel.png	420×900	1280×800	Re-shoot on a page background
docs/images/requests.png	420×1100	1280×800	Omit from v1 listing
docs/images/request-editor.png	420×1200	1280×800	Omit from v1 listing

Also required or recommended: at least one screenshot (five maximum), a 128×128 store icon (already have it), a 440×280 small promo tile, a category, a primary language, and a one-time \$5 developer registration fee.

What to do before publishing

Phase 1 — Remove the blockers

- ☐ **Write and host a privacy policy.** Cover: what is stored locally (settings, custom values, exclusions, API key), what is transmitted (field labels and constraints only, to the provider you chose), what is never transmitted (entered values, page URLs, page HTML), and that there is no Formly server and no analytics. GitHub Pages is sufficient hosting.
- ☐ **Produce a store build without `debugger`.** Remove the permission from the manifest and hide the Requests and Request Editor tabs. Keep the full build for local use.
- ☐ **Re-read the description against the single purpose rule.** The listing should describe one thing: filling forms with test data.
- ☐ **Add a `LICENSE` file** to the repository.

Phase 2 — Prepare the listing

- ☐ Create the developer account and pay the \$5 registration fee.
- ☐ Produce screenshots at exactly **1280×800**, showing: a form before filling, the same form after one click, and the Options page.
- ☐ Write the detailed description — what it does, who it is for, that data is fictional, that it never submits forms, and that AI is optional and uses the user's own key.
- ☐ Write a justification for every permission (drafts overleaf).
- ☐ Complete the data-use disclosures, and confirm they match the privacy policy word for word.

Phase 3 — Final checks before upload

- ☐ Run `npm run build`, then both test suites, on a clean checkout.
- ☐ Load the packaged `dist/` in a fresh Chrome profile and complete a full first run: install, pin, fill a form, open Options, set a key, fill again.
- ☐ Confirm the version number in `package.json` and `public/manifest.json` match, and raise it to `1.0.0`.
- ☐ Rotate any API key that has ever been pasted into a chat, commit, screenshot or test run.
- ☐ Zip the contents of `dist/` — the files at the archive root, not the folder itself.

Recommended scope for version 1.0

Submit the form filler alone. It is an obviously single-purpose extension with a modest permission set, which is a straightforward approval. Reintroduce the inspector later as a considered update with its own justification, once the listing has a review history — or keep it as the unpacked developer build, where no store policy applies.

FEATURE	VERSION 1.0	REASON
One-click filling, 42 field categories	Ship	The stated purpose
English / French / Arabic recognition	Ship	Core behaviour
Custom values and field exclusions	Ship	Configures the core behaviour
Side panel, field report, undo	Ship	Explains the core behaviour
Optional AI (Groq / Gemini, user key)	Ship	Improves generated values; disclose it
Request recorder	Hold	Requires <code>debugger</code> ; second purpose
Request editor and resend	Hold	Credential replay; third purpose

Appendix — permission justifications

The dashboard asks for one justification per permission. These drafts match what the code actually does; edit only if behaviour changes.

activeTab

"Formly fills the form on the tab the user is looking at. This permission grants access only to that tab and only when the user clicks the toolbar icon, which is the extension's single action."

scripting

"Filling a form requires injecting the fill routine into the current page. No script is injected until the user clicks the toolbar icon or the panel's Fill button."

storage

"Stores the user's own settings locally: generated-data language, custom field values, field exclusions, and an optional AI API key the user supplies. Nothing is synced and there is no remote server."

alarms

"Removes expired cached AI suggestions. Cached suggestions have a user-configurable expiry of 1 to 60 minutes and a browser alarm clears them when they lapse."

sidePanel

"Shows a panel beside the page listing which fields were filled or skipped and why, so the user can inspect a result, save a custom value, or exclude a field."

contextMenus

"Adds a right-click entry on the extension icon to open the Formly panel and the options page."

Host permission – generativelanguage.googleapis.com / api.groq.com

"When the user enables AI suggestions and supplies their own API key, Formly sends form field labels and constraints to the provider the user selected in order to generate relevant test values. Entered values, page URLs and page content are never sent."

Optional host permission – http://*/* and https://*/*

"Requested only if the user turns on 'Prepare ahead of the click', which generates suggestions as forms appear rather than waiting for a click. The user is asked at the moment they enable the setting and may decline; the extension works without it."

Formly 0.13.1 · Readiness audit performed against the built extension in `dist/` on 14 September 2026. Store policies and required asset dimensions change over time — confirm current requirements in the Chrome Web Store developer documentation before submitting.